# Chapter 2 Logging In and Using Basic SunOS Commands

This chapter describes the following topics.

- Logging In
- Logging Out
- Typing Commands
- Correcting Typing Mistakes
- Typing Multiple Commands and Long Commands
- Adding Command Options
- Using a Password
- Getting Help With OS Commands

To enter commands, use a terminal or window. Use your desktop environment documentation if you need information on starting a terminal or window.

# Logging In

A *standard work session* is the interval between the time you log in to the system and the time you log out. The SunOS multiuser environment requires that you identify yourself each time you want to use the system. Your *login name* (also known as a *user name* or an *account*) serves as your identity to the system and to other users on the system. Your *password* restricts use of your account to you. If you don't already have a login name and password, ask your *system administrator* to set up an account for you. After you have this information, you are ready to log in.

Before you log in to the system, your screen should look similar to the following:

```
hostname console login:
```

Type the login name given to you by the system administrator and press the Return key. For example, if your login name is *spanky*, type:

```
hostname console login: spanky
```

and press Return. Next, the system requests your password as follows:

```
hostname console login: spanky
Password:
```

Type your password at the prompt and press Return. If your account does not have a password assigned to it, the system logs you in without asking you for a password. Note that the system does not display (*echo*) your password on the screen as you type it. The failure to display your password helps to prevent others from discovering your password.

# Your Login Shell

In the chapters that follow, you enter SunOS commands. When you issue a command to the system, you are actually providing information to a command interpretation program, called a *shell*. The shell program then reads the information you have provided and causes the proper action to occur within the system.

The default shell for SunOS system software is the Bourne shell. The Solaris operating environment also supports the following shells.

- GNU Bourne Again shell (`bash`)
- C shell (`csh`)
- Korn shell (`ksh`)
- TC shell (`tcsh`)
- Z shell (`zsh`)

Each of these shells has its own unique differences.

---

**Note –**

You can get specific information about any SunOS command, including each of the available shells, by viewing its `man` (manual reference) page. For more information on `man` pages, see Displaying Manual Pages With `man`.

---

When you initially log in to the system (or open a new terminal or window) and you see your command prompt, it indicates that a shell program has been started for you automatically. This shell is called your *login shell*. If your login shell is not the SunOS default (the Bourne shell), it is because a different shell has been specified for you by your system administrator.

Some commands or procedures available when using one shell might not be available when using another shell. Unless stated otherwise, all commands and procedures described in this manual are available in the Bourne shell.

# Logging Out

When you have finished your work session and are ready to exit the operating system, log out by typing the following:

```
$ exit
```

After a moment, the system once again displays the login prompt:

```
$ exit
hostname console login:
```

When you see the login prompt, it indicates that you have successfully logged out. The system is now ready for you or another user to log in.

---

**Note –**

In the SunOS operating system, turning off your workstation or terminal does **not** necessarily log you out. Unless you log out explicitly, you might remain logged in to the system.

---

# Keyboard Equivalents

You can often speed up operations by using a sequence of keystrokes, called *keyboard accelerators*, that duplicate the operations of the mouse and menus, and of the preconfigured keyboard keys.

The following table lists several command operations and the keyboard equivalents for both SPARC™ and IA machines.

---

**Note –**

The Meta key is the <> key on SPARC keyboards and is obtained on IA keyboards by pressing Ctrl-Alt.

---

To use a keyboard accelerator, press and hold the first key (Meta or Ctrl-Alt together) and type the second key. For example, to cut selected text, press and hold the Meta key and press X on a SPARC system. On an IA system, press and hold Ctrl and Alt together and press X simultaneously.

Table 2–1 Keyboard Accelerators

| Operation | Keyboard Equivalent | Action |
|---|---|---|
| Again | Meta - a | Repeats the previous operation |
| Copy | Meta - c | Copies the selection to the clipboard |
| Cut | Meta - x | Cuts the selection and puts it on the clipboard |
| Find | Meta - f | Finds the selection to the right of the caret |
| Help | Help or F1 | Displays a help window with context-sensitive help for the object at the pointer location |
| New | Meta - n | Loads a new file |
| Open (File) | Meta - o | Opens a file (for example, if you've highlighted a file icon in File Manager) |
| Open (Window) | Meta - w | Opens an icon or closes a window to an icon |
| Paste | Meta - v | Copies the clipboard selection to the insertion point |
| Print | Meta - p | Sends the file to the printer (for example, if you've highlighted a file icon in File Manager) |
| Props | Meta - i | Displays the property window for the application at the pointer location |
| Redo | Shift-Meta - p | Undoes an Undo |
| Save | Meta - s | Saves the current file |
| Stop | Stop or Esc | Stops the current operation |
| Undo | Meta - u | Undoes the previous operation |

# Command Prompt

When you log in, the screen or window displays an initial prompt. The appearance of this prompt varies depending on the shell you are using and on how your system administrator originally set it up. Because the default command prompt for SunOS system software is the dollar sign (`$`), this prompt is used in most of the examples presented in this manual.

If you decide later that you want to change your command prompt, see Changing Your Command Prompt for instructions.

# Typing Commands

When you see the command prompt, the system is waiting for you to type a command. Try typing the command `date` at the prompt, as shown in this example (type `date` and press the Return key):

```
$ date
Mon Sep 17 10:12:51 PST 2001
$
```

This command displays the current date and time. When you type the same command, but capitalized, you receive the following message.

```
$ Date
Date: Command not found.
$
```

The Solaris operating environment interprets an uppercase `D` differently than a lowercase `d`, and the `Date` command fails. Most commands in the Solaris operating environment are lowercase.

## Correcting Typing Mistakes

The commands you type are not sent to the system until you press `Return`. If you type a command incorrectly, but do not press `Return`, you can correct your mistake in the following ways.

- Press the Delete or Back Space key to move back a space to the error; or
- Type Ctrl-U to erase the entire line and start over. Hold down the `Control` key and press `u`.

Try both of these methods and see how they work. The Delete/Back Space key varies on some systems. Ctrl-U should work on most systems.

## Typing Multiple Commands and Long Commands

You can type more than one command on a single line. Simply place a semicolon (`;`) between the commands, as shown here with the `date` command and the `logname` command:

```
$ date; logname
Tue Oct 31 15:16:00 MST 2000
spooky
```

This command entry displays the current date and time (from the `date` command) and the login name of the user currently logged in to the system (from the `logname` command).

If you are typing a long command, you can use the backslash character (\) to continue typing on a second line. For example:

```
$ date; \
logname
Tue Oct 31 15:17:30 MST 2000
spooky
```

Although the `date` and `logname` commands are not long commands, they demonstrate the concept of continuing a set of commands on the next line. Later, when the commands you want to use are longer than the width of your screen, you will see how useful the backslash character can be.

---

**Note –**

If you use a desktop window, you might not need to use the backslash character to continue typing commands on the next line. When you reach the end of a line, the commands you type wrap to the next line automatically, and the system executes all commands when you press `Return`.

---

## Repeating Previous Commands

The Korn, Bourne Again, C, TC, and Z shells enable your system to keep a **history** of commands you type and are able to repeat previous commands.

---

**Note –**

The Bourne shell (`sh`) does not support the `history` command.

---

### Repeating Commands in the Bourne Again, C, TC, or Z Shell

If you use the Bourne Again, C, TC, or Z shell, type `!!` and press Return to repeat the last command you typed.

```
example%!!
date
Tue Oct 31 15:18:38 MST 2000
example%
```

You can also repeat any previously typed command by typing `!`*x*, where *x* is the desired command's corresponding number on the **history list**. To see the history list, type the `history` command and press Return. The following is an example of what you might see.

```
example% history
1  pwd
2  clear
3  ls -l
4  cd $HOME
5  logname
6  date
7  history
```

---

**Note –**

The Z shell does not display the `history` command in the history list.

---

Another method for repeating characters from the history list is to follow the `!` with a negative number. For example, to repeat the second from the last command on the history list, type the following command.

```
example% !-2
date
Tue Oct 31 15:20:41 MST 2000
example%
```

---

**Note –**

If you use this command repetition method immediately after the history command in the Z shell, increase the negative number after the `!` by one (`!-3`).

---

Using the previous example history list, the `date` command is repeated.

You can also type the `!` character, followed by the first few characters of a previous command to repeat that command. For example, if you had previously typed the `clear` command to clear your screen, you could type `!cl` to clear your screen again. With this method for repeating commands, however, you must use enough characters for the desired command to be unique in the history list. If you use only one letter after the `!`, the system repeats the most recent command that begins with that letter.

### Repeating Commands in the Korn Shell

If you use the Korn shell, type the following command to repeat the previous command.

```
$ fc -s -
date
Tue Oct 31 15:18:38 MST 2000
$
```

You can also repeat any previously typed command by typing `fc -s` *x*, where *x* is the desired command's corresponding number on the **history list**. To see the history list, type the `fc -l` command and press Return. The following example is a sample history list.

```
$ fc -l
344  pwd
345  clear
346  ls -l
347  cd $HOME
348  logname
349  date
350  history
$
```

You can also repeat commands from the history list by following the `fc -s` command with a negative number. For example, to repeat the second from the last command on the history list, type the following command.

```
$ fc -s -2
date
Tue Oct 31 15:20:41 MST 2000
$
```

Using the previous example history list, the `date` command repeats.

You can also use the `fc -s` command with the first few characters of a previous command. For example, if you had previously typed the `date` command to display the current date, you could type `fc -s da` to display the date again. However, you must use enough characters for the desired command to be unique in the history list. If you use only one letter after the `fc -s` command, the system repeats the most recent command that begins with that letter.

## Adding Command Options

Many commands have **options** that invoke special features of the command. For example, the `date` command has the option **`-u`**, which expresses the date in Greenwich Mean Time instead of local time:

```
$ date -u
Tue Oct 31 22:33:16 GMT 2000
$
```

Most options are expressed as a single character preceded by a dash (`-`). Not all commands have options. Some commands have more than one option. If you use more than one option for a command, you can either type the options separately (**`-a -b`**) or together (**`-ab`**).

## Redirecting and Piping Command Output

Unless you indicate otherwise, commands normally display their results on the screen. Some special symbols allow you to **redirect** the output of a command. For example, you might want to save the output to a file rather than display it on the screen. The following example illustrates the use of the redirect symbol (>).

```
$ date > sample.file
$
```

In this example, the output from the `date` command is redirected to a new file called `sample.file`. You can display the contents of `sample.file` by typing the `more` command.

```
$ more sample.file
Tue Oct 31 15:34:45 MST 2000
$
```

As you can see, the contents of `sample.file` now contain the output from the `date` command. See Chapter 3, Working With Files and Directories for information on the `more` command.

Sometimes you might want to redirect the output of one command as input to another command. A set of commands strung together in this way is called a *pipeline*. The symbol for this type of redirection is a vertical bar (`|`) called a *pipe*.

For example, instead of saving the output of a command to a file, you might want to direct it as input to the command for printing (`lp`) by using the pipe symbol (`|`). To send the output from the `date` command directly to the printer, type the following:

```
$ date | lp
request id is jetprint-46 (1 file)
$
```

This pipeline would print the results of the `date` command. See Submitting Print Requests to the Default Printer for information on using the `lp` command to print files.

The command redirection examples shown here are simple, but when you learn more advanced commands, you will find many uses for piping and redirection.

## Running Commands in the Background

When you type a command and press the Return key, your system runs the command, waits for the command to complete a task, and then prompts you for another command. However, some commands can take a long time to finish, and you might prefer to type other commands in the meantime. If you want to run additional commands while a previous command runs, you can run a command in the **background**.

If you know you want to run a command in the background, type an ampersand (`&`) after the command as shown in the following example.

```
$ bigjob &
[1] 7493
$
```

The number that follows is the process id. The command `bigjob` will now run in the background, and you can continue to type other commands. After the job completes, you will see a message similar to the following the next time you type another command, such as `date` in the following example.

```
$ date
Tue Oct 31 15:44:59 MST 2000
[1]    Done                  bigjob
$
```

If you plan to log off before a background job completes, use the `nohup` (no hangup) command to enable the job to complete, as shown in the following example. If you do not use the `nohup` command , the background job terminates when you log off.

```
$ nohup bigjob &
[3] 7495
$
```

# Using a Password

To ensure your system's security, the Solaris operating environment requires you use a password to access your system. Changing your password several times a year helps to ensure that you are the only user with easy access to your account.

---

**Note –**

If you believe someone has used your account without your permission, change your password immediately.

---

When you choose a password, follow these guidelines.

- Choose a password that you can remember without writing it down. A password that you cannot remember is worse than one that is too easily guessed.
- Choose a password that is at least six characters long and contains at least one number.
- Don't use your own name or initials or the name or initials of your spouse.
- Don't use the names of pets or objects common to your interests.
- Don't use all capital letters.
- If you have more than one account, don't use the same password for every account.
- Avoid using the characters Ctrl-C, Ctrl-Z, Ctrl-U, Ctrl-S, Esc, Tab, #, and @ in your password. The terminal might interpret these characters as signals rather than text characters, and this interpretation would preclude you from properly typing in your password.

## Changing Your Password

To change your personal password, type the `passwd` command:

```
$ passwd
passwd:  Changing password for user2
Enter login password:
New password:
Re-enter new password:
passwd (SYSTEM): passwd successfully changed for user2
$
```

1. When the system prompts you for `Enter login password:`, type your current password.

   If no password is currently assigned to your account, the system skips the `Old Password:` prompt.

   The system does not echo (display) your password on the screen and thereby prevents other users from discovering your password.
2. When the system prompts you for `New Password:`, type your new password.

   Again, the password you type does not echo on the screen.
3. At the final prompt, `Re-enter new password:`, type your new password a second time.

   Your system verifies that you typed the password you intended to type.

   If you do not type your password precisely the way you did at the previous prompt, the system refuses to change your password and responds with the following message:

   ```
   passwd: They don't match; try again.
   ```

   If you receive this message repeatedly, contact your system administrator to get a new password.

   ---

   **Note –**

   Passwords containing fewer than six characters are not allowed. Also, a new password must differ from the old password by at least three characters.

   ---

## Password Aging

If your system uses password aging (implemented with options to the `passwd` command), your password can have either a maximum, or a maximum **and** minimum lifespan. The lifespan of your password is set by your system administrator.

When your password reaches the maturity date, your system prompts you to change your password when you log in. The following message displays.

```
Your password has expired. Choose a new one.
```

The system then automatically runs the `passwd` program and prompts you for a new password.

If, for example, the **minimum** age of your password has been set for two weeks, and you try to change your password before the minimum life span has elapsed, the following message displays.

```
Sorry, less than 2 weeks since the last change.
```

For more information on passwd(1) and password aging, refer to the *man Pages(1): User Commands*.

# Getting Help With OS Commands

This section describes various online help features. These features enable you to view reference information from your workstation or terminal.

## Displaying Manual Pages With `man`

If you know the name of a command, but you are not sure what it does, the `man` command can be helpful. Type the following to find out more about this command:

```
$ man man
```

This command displays the first part of a SunOS manual reference page in the window display area. Press the space bar to see the next screen, or press the Q key to quit and return to the command prompt. Use the `man` command to see all the available options and to show the proper command syntax. Manual reference pages often provide examples that illustrate various uses of the command.

## Displaying a One-line Summary With `whatis`

If you want just a one-line summary of the command's function, use the `whatis` command, as shown here:

```
$ whatis date
date (1)          -display or set the date
$
```

The number in parentheses after the command name in the previous example indicates the Reference Manual section to which this command belongs. Commands are grouped into various categories according to function. Most user commands are in section 1. By common convention, the section number is displayed in parentheses after the name of the command. You can find the printed manual reference page for a command in alphabetical order within its group.

---

**Note –**

The `whatis` command is only available if your system administrator has set up a special database of command descriptions.

---

## Keyword Lookup With `apropos`

If you know what you want to do, but you do not know which command to use, use the `apropos` command to locate commands by keyword lookup. The `apropos` command lists all commands that have one-line summaries that contain any keywords you supply. The output of the `apropos` can be lengthy, as some keywords might appear in many places.

---

**Note –**

The `apropos` command is only available if your system administrator has set up a special database of command descriptions.

---

To view examples of `apropos` output, type one or more of the following commands.

- `apropos who`
- `apropos execute`
- `apropos apropos`

If you do type a keyword that generates an unreasonably lengthy display, press Ctrl-C to interrupt the display and return you to the command prompt. Hold down the Control key and press “c.”

- © 2010, Oracle Corporation and/or its affiliates